SCHOOL OF NATURAL AND APPLIED SCIENCE

DEPARTMENT OF COMPUTING

TO : Mr. S. YUTE

FROM : SIMON MHONE & TIYAMIKE NGOMA

REGISTRATION NO : BSC/29/23 & BSC/COM/NE/24/23

COURSE TITLE : NETWORK PROGRAMMING AND APPLICATION DEVELOPMENT

COURSE CODE : NET 322

ASSIGNMENT TITLE : PLANNING & DEVELOPMENT ARCHITECTURE

DUE DATE : 05th MAY, 2026

Introduction

This document describes the architecture, planning, communication protocols, concurrency model, and failure handling mechanisms of a Real-Time Sensor Monitoring System.

The system is designed to:

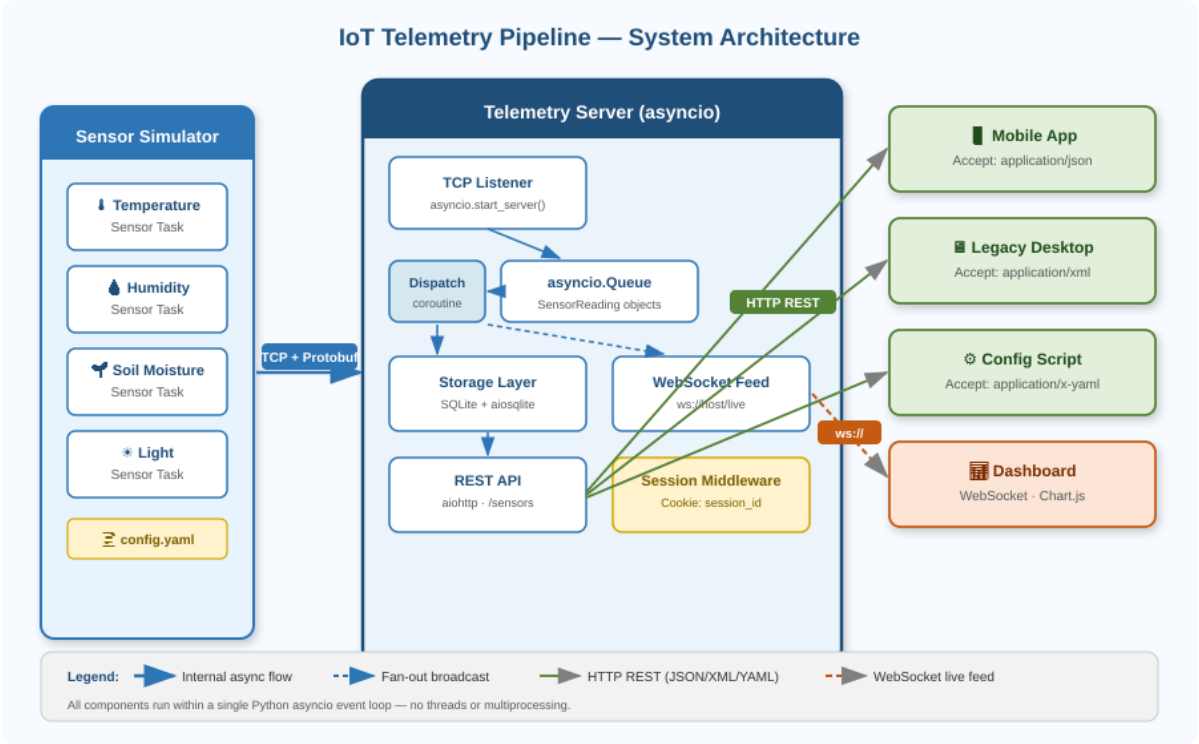
- Collect sensor readings from distributed sensor nodes
- Serialize data using Protocol Buffers
- Transmit readings to a central server using TCP
- Persist readings in a storage layer
- Expose REST APIs for historical data access
- Broadcast live updates to dashboards using WebSockets
- Support asynchronous and scalable communication using Python AsyncIO

The design emphasizes:

- Scalability
- Fault tolerance
- Low latency
- Efficient serialization
- Concurrent processing
- Real-time streaming

A. System Architecture Diagram

Architecture Overview

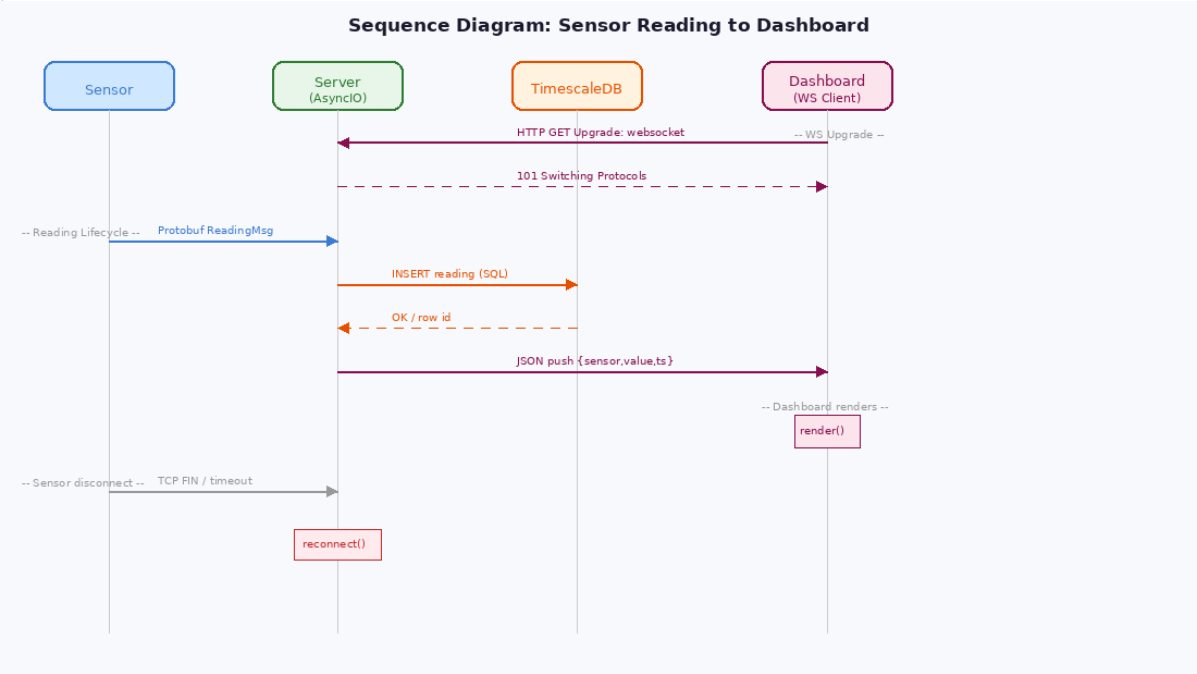


Communication Protocols and Serialization Formats

Source	Destination	Protocol	Serialization Format
Sensor Client	Central Server	TCP	Protocol Buffers
REST Client	REST API Server	HTTP/HTTPS	JSON
Web Dashboard	WebSocket Hub	WebSocket	JSON
Server	Database	SQL	Relational Schema
Config File	Sensor Client	File I/O	YAML

B. Sequence Diagram

Full Lifecycle: Sensor Reading to Live Dashboard



WebSocket Upgrade Handshake

Client Request

```
GET /live HTTP/1.1
Host: localhost:8000
 Upgrade: websocket
 Connection: Upgrade
 Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Server Response

```
HTTP/1.1 101 Switching Protocols
 Upgrade: websocket
 Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

After the handshake completes, the TCP connection becomes a full-duplex WebSocket connection used for live sensor updates. The server can now push JSON-encoded ReadingEvent messages to the dashboard at any time without the client polling.

C. Data Model

Sensor Schema

Field	type	Description
id	string	Unique sensor identifier
name	string	Human-readable name
type	string	Sensor type
location	string	Physical location
interval	double	Time interval in seconds between consecutive sensor reading sent to the server

Reading Schema

field	type	Description
Sensor_id	string	Associated sensor
type	string	Sensor type
value	double	Measured value
Timestamp	double	Reading timestamp
Unit	string	Unit of measurement

Protocol Buffers Definition

```
syntax = "proto3";

package telemetry;

message Reading {

 string sensor_id    = 1;  // unique sensor identifier

 string type         = 2;  // temperature | humidity | soil_moisture | light

 double value        = 3;  // numeric measurement

 double timestamp    = 4;  // Unix epoch seconds (float)
```

```
  string unit         = 5;  // e.g. "°C", "%", "lux"

}



message SensorRegistration {

  string sensor_id        = 1;

  string type             = 2;

  string location         = 3;

  double interval_seconds  = 4;

}
```

D. REST Endpoint Specification

Endpoint 1 List all Sensors

Property	Value
Path	/sensors
Method	GET
Request Body	NONE
Response Body	Array of sensor objects (JSON / XML / YAML per Accept header)
Success Status	Created
Error Status	400 Bad Request

EXAMPLE

GET /sensors HTTP/1.1 Accept: application/json Cookie: telemetry_session=abc123

< 200 OK < Content-Type: application/json < Set-Cookie: telemetry_session=abc123; HttpOnly

[{ "sensor_id": "temp_gh1","type": "temperature", "unit": "degC", "location": "Greenhouse 1", "interval_s": 10, "active": true }]

Endpoint 2 Get Sensor Readings

Property	Value
Path	`/Sensors`
Method	`POST`
Request Body	None
Response Body	{"sensor_id": "...", "name": "...", "type": "...", "unit": "...", "location": "...", "interval_s": 10}
Success Status	201 created
Error status	404 not found

EXAMPLE

```
http
```

POST /sensors HTTP/1.1 Content-Type: application/json Cookie: telemetry_session=abc123

{ "sensor_id": "temp_gh1", "name": "type": "temperature", "unit": "C", "location": "Greenhouse 1 ZONE A", "interval_s": 5 }

< 201 Created < Location: /sensors/temp_gh1

{"sensor_id":"temp_gh1", ...}

E. YAML Configuration Schema

Configuration Structure

The sensor client loads its configuration from a YAML file during startup.

Schema

Field	Type	Description
sensor.id	String	Unique sensor ID
sensor.type	String	Sensor type
sensor.location	String	Deployment location
server.host	String	Server IP/domain
server.port	Integer	TCP server port
interval_seconds	Integer	Publishing interval

Worked Example

sensor: id: temp_gh1
type: temperature
location: Greenhouse 1- Zone A

server:

host: 127.0.0.1

port: 9000

interval_seconds: 4

F. Concurrency Model

AsyncIO Task Organization

The server is implemented using Python AsyncIO.

The application uses a single event loop with multiple asynchronous coroutines running concurrently.

Main Coroutines

1. TCP Listener Coroutine

Responsibilities:

- Accept incoming sensor TCP connections

- Spawn per-client handler coroutines
- Maintain active connection registry Communication:

- Passes decoded messages into an AsyncIO queue

2. Sensor Client Handler Coroutine

Responsibilities:

- Receive Protocol Buffer messages
- Decode sensor readings
- Validate message structure
- Push readings to processing queue Communication:

- Sends validated readings to the storage coroutine
- Sends live events to the WebSocket broadcaster

3. Storage Coroutine

Responsibilities:

- Persist readings into database
- Retry transient failures
- Batch inserts when necessary, Communication:

- Consumes readings from AsyncIO queue

4. REST API Coroutine

Responsibilities:

- Handle HTTP requests
- Query historical data
- Return JSON responses Communication:

- Reads from database layer

5. WebSocket Broadcaster Coroutine

Responsibilities:

- Maintain dashboard client connections
- Broadcast live sensor readings
- Remove disconnected clients Communication:

- Consumes live events from queue

Inter-Coroutine Communication

The coroutines communicate using:

- AsyncIO queues
- Shared connection registries
- Awaitable tasks
- Event-driven callbacks Example:

```
TCP Receiver -> asyncio.Queue(500) -> Storage Coroutine -> SQLite | v Per-conn Queue(50) x N clients | v WebSocket Broadcaster -> Dashboard
```

Handling Slow Consumers

A slow WebSocket consumer may fail to process incoming updates quickly enough.
The system handles this using:

1. Bounded outgoing queues per client
2. Backpressure management
3. Connection termination for severely lagging clients Behavior:

- If a client's queue becomes full, old messages may be dropped.
- If the client remains slow for too long, the server disconnects it.
- Other clients continue operating normally.

This prevents one slow dashboard from blocking the entire system.

G. Failure Handling

1. Sensor Disconnects

Detection

The server detects disconnects when:

- TCP socket closes
- Read timeout occurs
- Heartbeat messages stop arriving

Server Actions

- Mark sensor as offline
- Remove connection from active registry
- Log the disconnect event
- Continue serving other sensors

Recovery

Sensors automatically reconnect using retry logic with exponential backoff.

---

2. Storage Layer Failure

Possible Failures

- Database unavailable
- Disk full
- Query timeout
- Connection pool exhaustion

System Behavior

- Readings are temporarily buffered in memory queues
- Retry attempts are made asynchronously
- Failed writes are logged
- Alerts may be triggered

Overflow Protection

If the buffer becomes full:

- Oldest messages may be discarded
- Server enters degraded mode
- Rate limiting may activate

---

3. Server Overload

Causes

- Excessive sensor traffic
- Too many dashboard connections

- CPU exhaustion
- Memory pressure

Protection Mechanisms

Rate Limiting

Restricts excessive client requests.

Backpressure

Slows producers when queues become full.

Queue Limits

Prevents unbounded memory growth.

Connection Shedding

Low-priority or slow clients may be disconnected.

Load Monitoring

The server continuously monitors:

- Queue sizes
- CPU usage
- Memory usage
- Active connections

---

10. Security Considerations

Authentication

REST and WebSocket endpoints may require API tokens.

Encryption

TLS can be enabled for:

- HTTPS communication
- Secure WebSocket connections (WSS)
- Sensor TCP communication

Validation

All incoming messages are validated to prevent malformed payloads.

. Scalability Considerations

The architecture supports horizontal scaling by:

- Running multiple server instances
- Using a shared database
- Deploying a message broker
- Load balancing WebSocket traffic Potential improvements:

- Kafka/RabbitMQ integration
- Distributed caching
- Container orchestration with Kubernetes

---

12. Conclusion

This architecture provides a scalable and fault-tolerant platform for real-time sensor monitoring.

Key characteristics include:

- Efficient binary serialization with Protocol Buffers
- Real-time updates using WebSockets
- Asynchronous concurrency using AsyncIO
- Persistent storage for historical analysis
- Failure isolation and recovery mechanisms
- REST APIs for interoperability

The design ensures that the system remains responsive, reliable, and scalable even under high load conditions.